CHAPTER 11: ARRAY SORTING

Arranging data in an ordered sequence is called "sorting". This order can be done in two ways: ascending and descending.

- Ascending (or increasing) order: arranged from the smallest to the largest element.
- Descending (or decreasing/non-increasing) order: arranged from the largest to the smallest element.

Java has a built-in Arrays.sort() method for sorting arrays. Let's see that first.

Code	Output

import java.util.Arrays; // importing Arrays class
public class BuiltInSortAscending{
 	
public static void main(String[] args){
int[] arr = { 10, -5, 7, 45, 27 };
 	
// calling the built-in sorting method on the created array
Arrays.sort(arr);
 	
// displaying the elements of the sorted array
for (int elem = 0; elem < arr.length; elem++){
System.out.println(arr[elem]);
}
}
}
	-5
	7
	10
	27
	45

The above code uses the Java built-in method to sort an array in ascending order. Now, if you just add another step and reverse the sorted array, you will get the sorted array in descending order.

But this is pretty straight forward. As a Computer Science student, you need to know the underlying concepts behind array sorting. You need to understand and visualize different sorting techniques. There are a lot of sorting algorithms used in several domains. In this course, we will learn three such common sorting algorithms.

BRAC UNIVERSITY
Inspiring Excellence

11.1 SELECTION SORT

The first technique in our list is called Selection Sort. Here, in the ascending order, at first we need to find the minimum value of the given array and swap (exchange) it with the first element of the array. Then, we need to find the second minimum from the rest of the array elements (starting from the 2nd index) and swap it with the 2nd element of the array. Then, we need to find the 3rd minimum from the rest of the array elements (starting from the 3rd index) and swap it with the 3rd element of the array. We need to continue this process until all the elements of the array have moved to their proper position.

At each iteration, the array is partitioned into two sections. **The left section is sorted and the right section is being processed.** Each iteration, we move the partition one step to the right, until the entire array elements have been processed.

Let's consider the following array.

element	40	2	27	-7	14
index	**0**	**1**	**2**	**3**	**4**

Now, let's visualize how Selection Sort works. We will sort the array in ascending order.

Iteration 1: At first, we will consider the element in index 0 as the minimum value which is 40 in this example. Then, we will search from index 1 to index (length-1) to find an element less than 40.

element	40	2	27	-7	14
index	**0**	**1**	**2**	**3**	**4**

Here, we have found the lowest possible value less than 40 in index 3 which is -7. We will swap the values of these two indexes.

element	-7	2	27	40	14
index	**0**	**1**	**2**	**3**	**4**

After swapping, we can see that -7 is in the correct sorted position.

Iteration 2: Here, we will consider the element in index 1 as the minimum value which is 2. Then, we will search from index 2 to index (length-1) to find an element less than 2.

element	-7	2	27	40	14
index	**0**	**1**	**2**	**3**	**4**

But there are no values which are less than 2 from index 2 to 4; which means 2 is already in the correct sorted position. So, no swapping will be needed here.

element	-7	2	27	40	14
index	**0**	**1**	**2**	**3**	**4**

Position of element 2 will be unchanged here.

Iteration 3: You guessed it right. Now, we will look for the value smaller than 27 from index 3 to 4.

element	-7	2	27	40	14
index	**0**	**1**	**2**	**3**	**4**

The value less than 27 is 14 which is in index 4. So, we will swap the values of index 2 and 4.

element	-7	2	14	40	27
index	**0**	**1**	**2**	**3**	**4**

Iteration 4: Similarly, the value of index 3 and 4 will be swapped since 27 is less than 40.

element	-7	2	14	27	40
index	**0**	**1**	**2**	**3**	**4**

Iteration 5: This step is kind of useless and unnecessary. Because all the remaining elements are sorted from left to right. So, the element in the last index of the array will definitely be in the correct sorted position. So, the position of 40 will be unchanged.

element	-7	2	14	27	40

index	0	1	2	3	4

You can see our array is now successfully sorted in the ascending order using Selection Sort. For descending order sort, just find the maximum value instead of the minimum value on each iteration.

Here, for each iteration, we only consider the right section of the partition for finding the minimum value as the left side is already sorted. Lastly, in the last step, only one element remains which is the minimum. So sorting is unnecessary in that case. So, for Selection Sort, (length-1) times iteration is needed.

Now, let's look at the code.

Code	Output

```
public class SelectionSortAscending{

   public static void main(String[] args){
      int[] arr = { 40, 2, 27, -7, 14 };

       // implementing Selection Sort on the created array

      for (int i = 0; i < arr.length-1; i++){
         int min_idx = i;
         // finding out the index containing minimum value from i+1 to last index
         for (int j = i+1; j < arr.length; j++){
            if (arr[j] < arr[min_idx]){
               min_idx = j;
            }
         }
         // swapping the minimum index value with the current index value
         int temp = arr[min_idx];
         arr[min_idx] = arr[i];
         arr[i] = temp;
      }

      // displaying the elements of the sorted array
      for (int idx = 0; idx < arr.length; idx++){
         System.out.println(arr[idx]);
      }
   }
}
```

```
-7
2
14
27
40
```

11.2 BUBBLE SORT

Now, we will learn the Bubble Sort technique. For Bubble sort, in each iteration, adjacent elements are compared. If the adjacent elements are in the wrong order, then they are swapped. It has been named "Bubble sort" because of the way smaller or larger elements "bubble" to the top (Left side) of the array. As the number of iterations increases, the number of comparisons decreases.

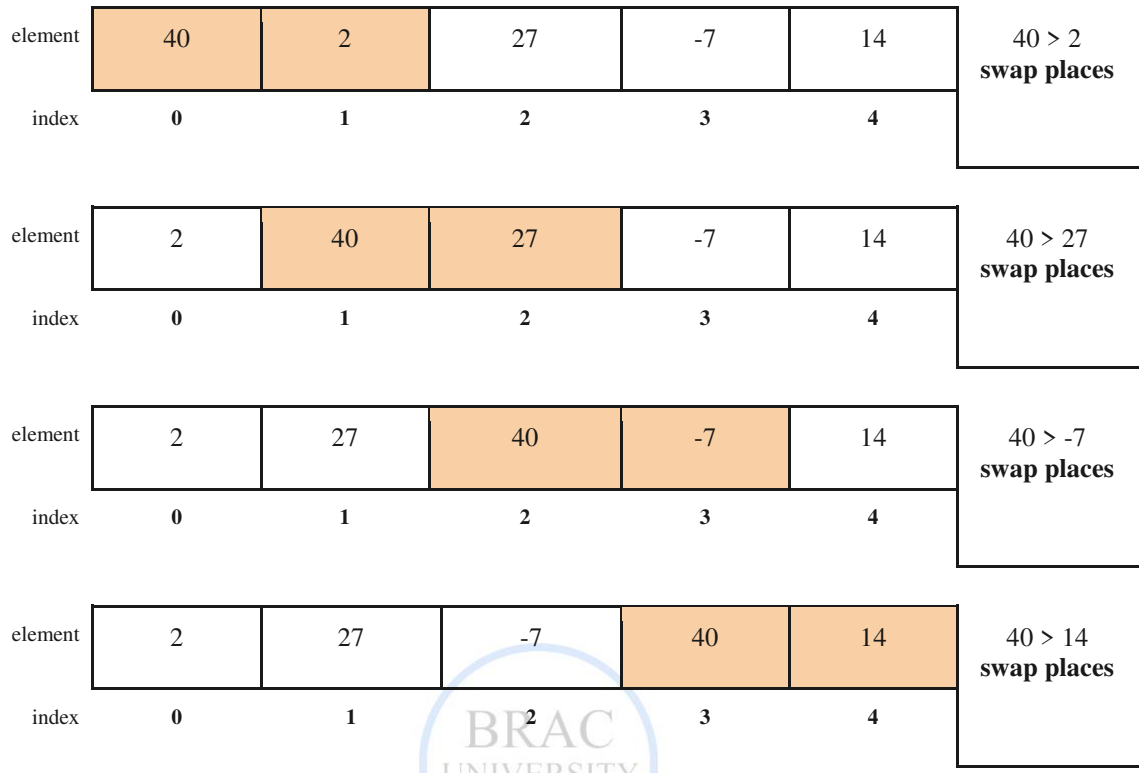
Let's consider the same array as before.

element	40	2	27	-7	14
index	0	1	2	3	4

Let's visualize how this algorithm works. Here, we will sort the array in ascending order too.

Iteration 1: We will compare between two adjacent index elements here. If the first element is greater than the second element, then we will swap between two index values. For example, we will compare the index 0 value

with index 1 value, which means we will compare between 40 and 2. Since, 2 is less than 40, so these two values will be swapped. Now, 40 is in index 1. We will compare 40 with the index 2 element which is 27. Again, these two values will be swapped and 40 will move to index 3. Similarly, we need to do the same process for the entire array in this iteration.

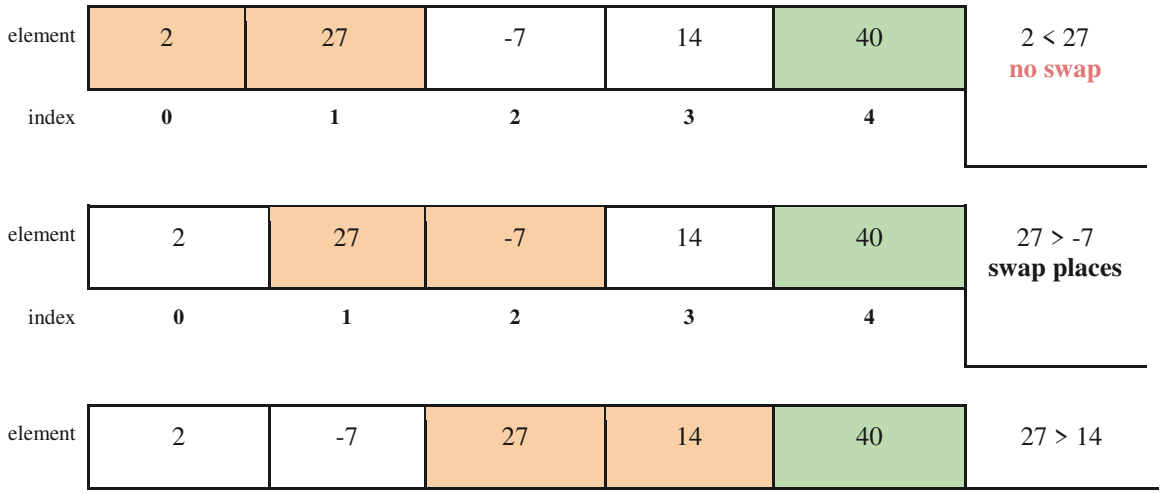


BRAC UNIVERSITY
Inspiring Excellence

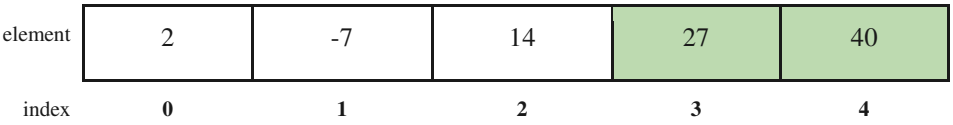
After these **4** comparisons, the array looks like this:

You can see after iteration 1, the largest element 40 moved to the last index position. So, 40 is now in the sorted position and we do not need to check for 40 again in the upcoming iterations.

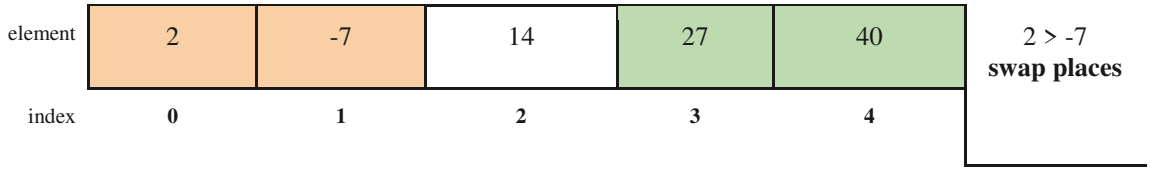
Iteration 2: We will follow the similar procedure for the remaining unsorted array elements.

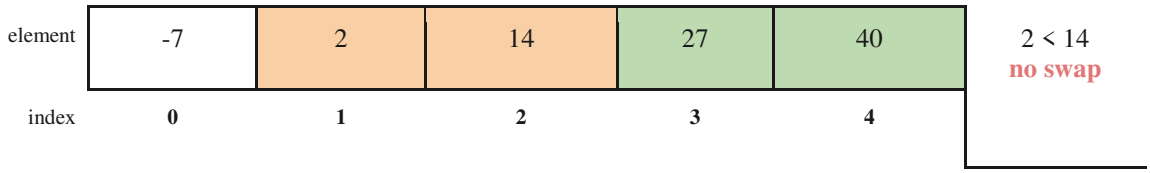


After these **3** comparisons, the array looks like below:

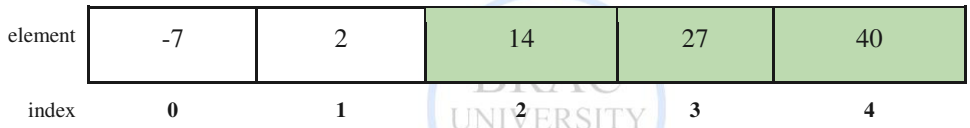


Iteration 3: You get the idea now. We will now perform **2** comparisons.



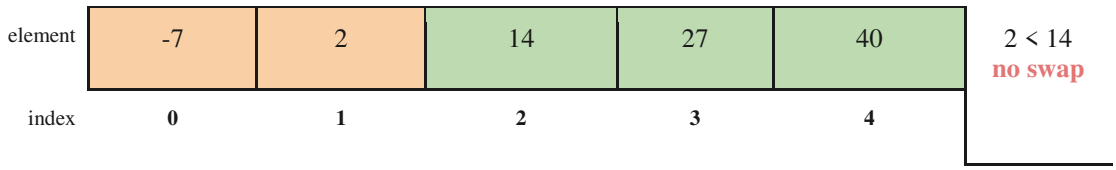


After the iteration, now the array looks like below:

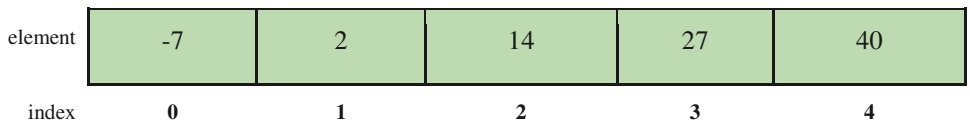


We can see the array is already sorted. But the algorithm will run for another iteration.

Iteration 4: The algorithm will compare between -7 and 2. But there will be no swap because -7 is less than 2.



So, -7 and 2 are in the correct sorted position. So, after the 4th iteration, we get the sorted array.



Similar to Selection Sort, for Bubble Sort, (length-1) times iteration is needed. Another thing to notice is that here in each iteration, the number of comparisons is reduced by one. For example, in our example, in the 1st iteration, there were 4 comparisons. In the 2nd iteration, there were 3 comparisons and so on.

For the descending order sort, we just need to compare the two adjacent index values and if the later element is greater than its immediate previous element, then the two array elements will be swapped.

Now, let's look at the code.

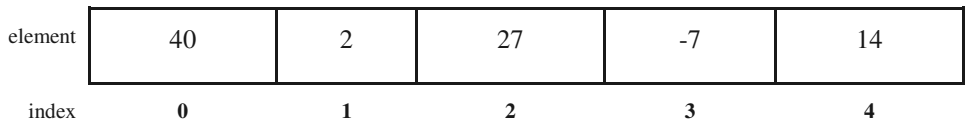
Code	Output

public class BubbleSortAscending{
 	
  public static void main(String[] args){
  int[] arr = { 40, 2, 27, -7, 14 };
 	
  // implementing Bubble Sort on the created array
 	
  for (int i = 0; i < arr.length-1; i++){
  // comparing between the two adjacent index elements in a loop
  for (int j = 0; j < arr.length-i-1; j++){
  if (arr[j+1] < arr[j]){
  // swapping the values
  int temp = arr[j];
  arr[j] = arr[j+1];
  arr[j+1] = temp;
  }
  }
  }
 	
  // displaying the elements of the sorted array
  for (int idx = 0; idx < arr.length; idx++){
  System.out.println(arr[idx]);
  }
  }
}
	-7
	2
	14
	27
	40

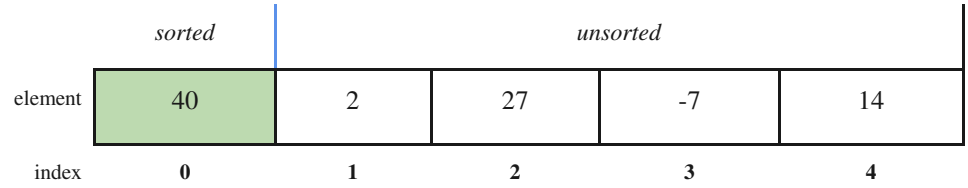
11.3 INSERTION SORT

The last sorting technique we will learn in this chapter is called Insertion Sort. It has a similarity with sorting playing cards by hand. The first card in the card game is expected to be sorted, and afterwards we choose an unsorted card. If the unsorted card chosen is less than the first, it will be placed on the left side; otherwise, it will be placed on the right side. Similarly, all unsorted cards are taken and placed exactly where they belong. So, in this technique, the array effectively breaks down itself into two sections: sorted and unsorted. We select a value from the unsorted section one by one and place that value in the correct position in the sorted section.

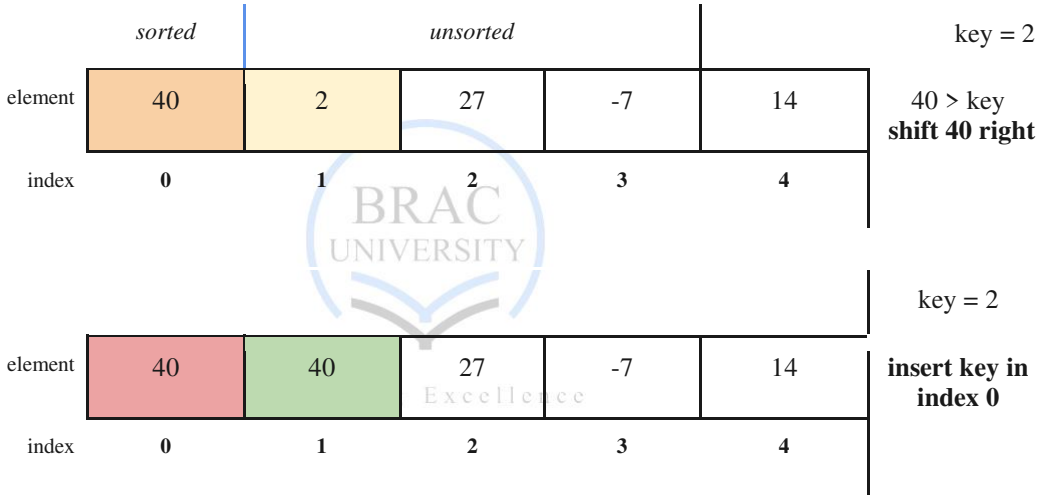
Let's visualize the working procedures of this technique using the same array used before.



Initially we assume that the first index element is already sorted. So, as mentioned earlier, the array can be split into two parts like below:

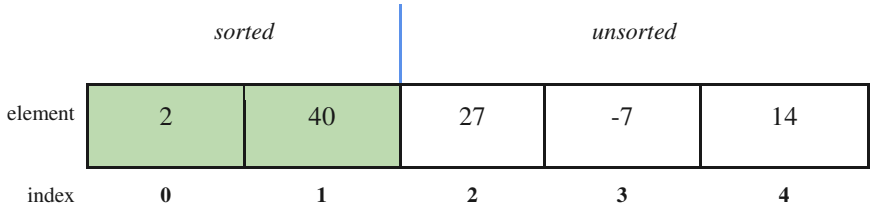


Iteration 1: We consider the value of index 1 as key. So, key = 2 in this case. Here, key means the value of the current array element which will be inserted in the correct position in the sorted section. Here, the value of index 0 is 40, which is greater than the key value 2. So, 40 will be moved to index 1 and the value of key will be inserted to index 0.

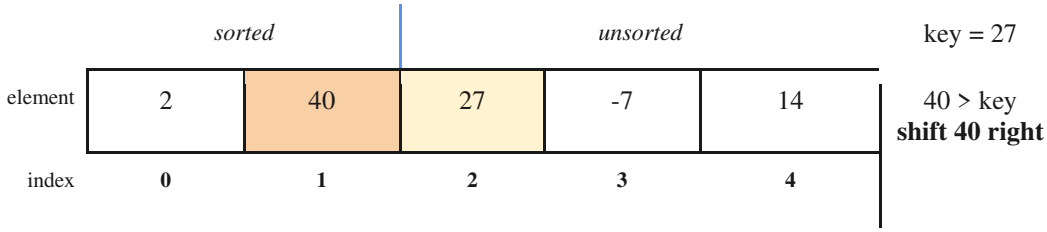


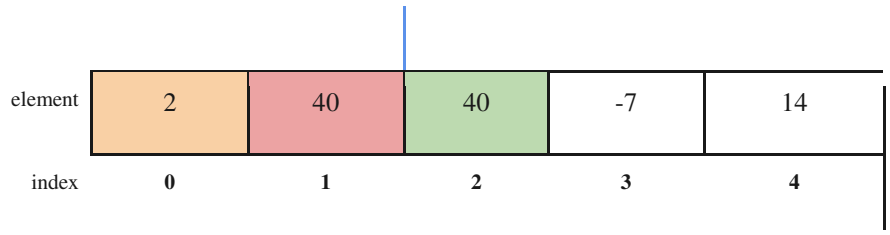
BRAC UNIVERSITY
Excellence

After inserting the key in index 0, the array now looks like this.

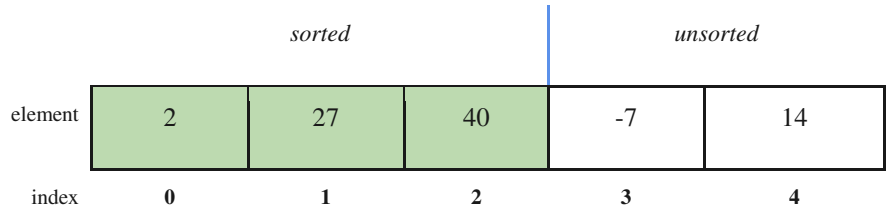


Iteration 2: The value of key will be the value of index 2 now. So, key = 27. We need to follow the same procedure as before.

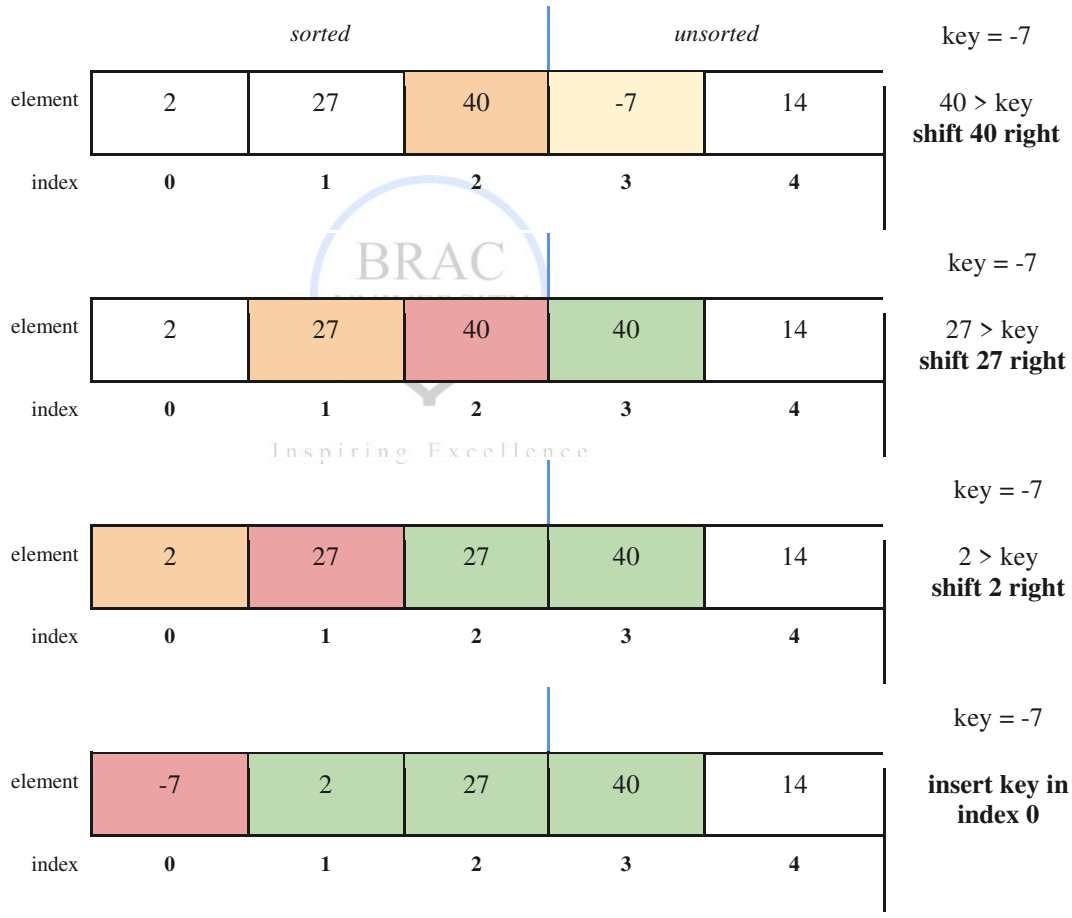




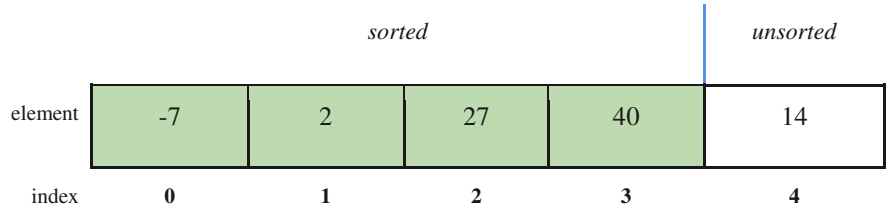
After inserting the key in index 1, the array looks like below:



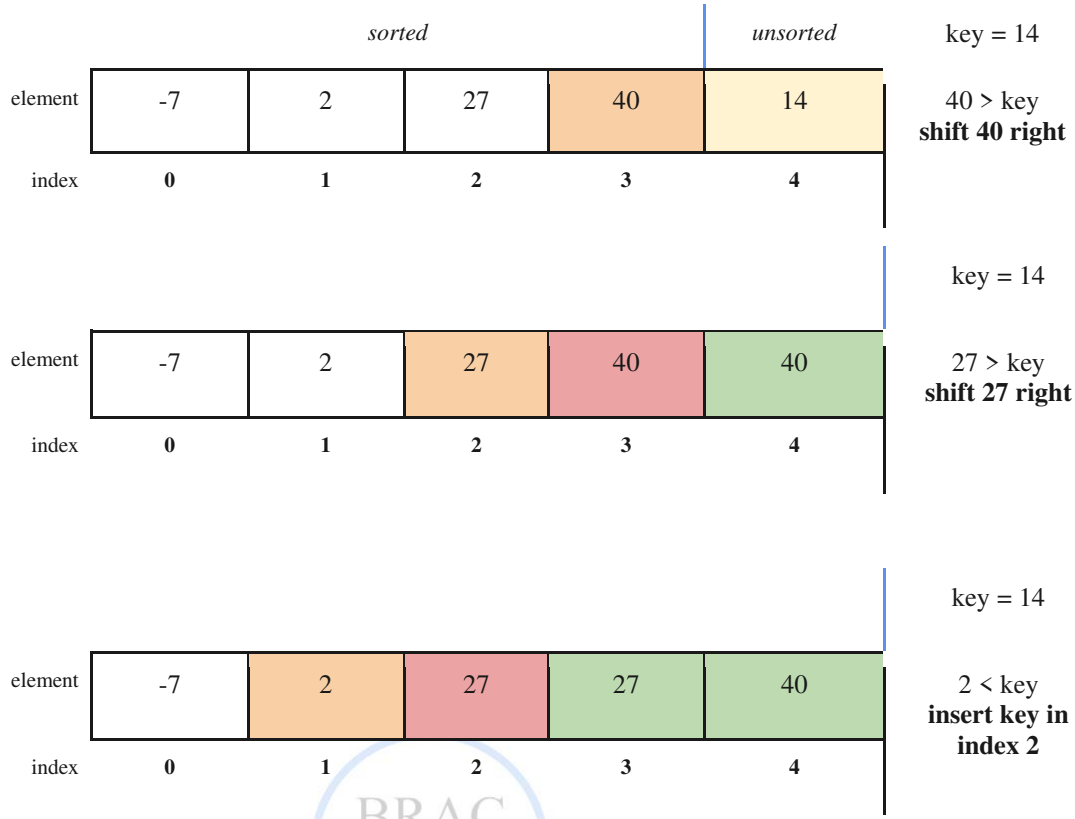
Iteration 3: The value of the key will be -7 now. So, key = -7.



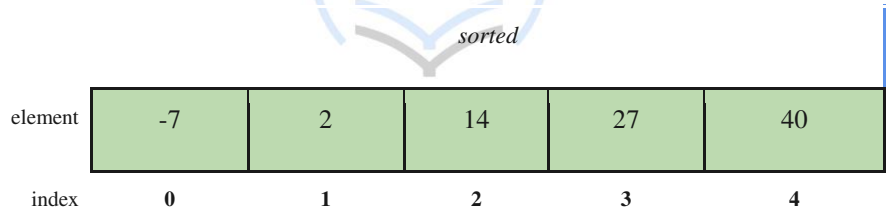
After inserting the key, the array will look like this:



Iteration 4: We now have to insert 14 in the correct position in this step. Here, key = 14.



After inserting the key in index 2, we now have the full array sorted.



That's how the Insertion sort algorithm works. Here, we initially assume the index 0 value is already in the sorted position. So, we look from index 1 to the last index of the array. So, (length-1) times iteration is needed here too. We consider each index value as the key and insert that key into the appropriate position in the sorted part of the array.

For the descending order sort, we will shift the current index value if it is less than the key and insert the key in the appropriate position in each step afterwards.

Now, let us look into the code.

Code	Output

```
public class InsertionSortAscending{

   public static void main(String[] args){
      int[] arr = { 40, 2, 27, -7, 14 };

       // implementing Insertion Sort on the created array

      for (int i = 1; i < arr.length; i++){
         int key = arr[i]; // the element which will be inserted
         int j = i - 1;
         /* shift elements of index 0 to (i-1) which are greater
         than key to one index forward of the current index */
         while (j >= 0 && arr[j] > key){
            arr[j + 1] = arr[j];
            j = j - 1;
         }
         arr[j + 1] = key; // inserting the key in the suitable place
      }

      // displaying the elements of the sorted array
      for (int idx = 0; idx < arr.length; idx++){
         System.out.println(arr[idx]);
      }
   }
}
```

```
-7
2
14
27
40
```

BRAC
UNIVERSITY
Inspiring Excellence

11.4 WORKSHEET

A. Using Java built-in Arrays.sort() method, sort an array in descending order. You cannot use any other built-in method to complete this task. [Hint: Modify the ascending order sort code and add another step using a loop.]

B. Implement the descending order sort for the following sorting techniques:

1. Selection Sort,
2. Bubble Sort,
3. Insertion Sort.

A. We have learnt that Bubble sort needs (length - 1) times iteration. But sometimes even before completing all the iterations, the array might get sorted and the code runs for some extra redundant iterations. In that case, modify your ascending/descending order Bubble sort code so that it can handle this issue and stop immediately without running for these redundant iterations. [Hint: Utilize the concept of Flag variable.]

B. Let's say you have an array of n elements. You only need the first k sorted elements (where k < n) of the array. Which sorting algorithm will be efficient in this case? Share your opinion briefly.

C. Suppose, you have an array whose elements are nearly sorted (for example, 90% of the elements are already in the correct sorting position). To sort the remaining values faster, which sorting technique will you use? Explain shortly.

D. We have seen in both Selection Sort and Bubble Sort; we need to perform swapping. If we want to minimize the overall swap operations, which sorting technique would be better? Explain briefly.

E. Write a Java program that takes a number T as user input where T denotes the total number of students. Then the user needs to take the name and corresponding marks of T students as input. Display the names of the students based on their marks from highest to lowest.

Sample Input	Sample Output
5 Clark 56 Bruce 51 Diana 76 Barry 91 Lois 63	Barry Diana Lois Clark Bruce
Explanation: In this sample example, T=5. So, the user needs to enter 5 students' names and marks one by one as input. From the sample, Clark got 56, Bruce got 51, and so on. In the output, the student names are displayed based on the marks they have received in descending order.	